Topic 8 Midterm Project

1. Description (5 points)

In modern organizational workflows, tracking projects, departmental structures, task progress, and employee working hours using manual methods or fragmented spreadsheets creates data redundancy, input errors, and a lack of real-time visibility.

This database is designed to establish a centralized, reliable relational environment that unifies project portfolio management, resource allocation, and cost tracking. By capturing employee hourly rates (hourly_rate), billable hours (is_billable), task completion metrics (completion_pct), and manager-employee reporting chains, the database transitions the organization from simple activity recording to data-driven financial and operational analytics. Management can evaluate actual labor expenses against project budgets in real time, prevent task bottlenecks, and optimize staff utilization.

1. Supported Tasks, Operations, Processes, and User Groups Served

Supported tasks, operations, and processes:

The system provides structural and transactional support for the following core business processes:

1. Departmental & Organizational Management:
    - Maintaining department structures, office locations, and allocated budgets.
    - Tracking reporting hierarchies between team members and their managers via self-referencing relationships.
2. Project Lifecycle & Budget Control:
    - Initializing projects with unique system codes, client names, and strict start/end date constraints.
    - Setting approved budgets and monitoring lifecycle states (Planning, In Progress, On Hold, Completed, Cancelled).
3. Task Assignment & Progress Tracking:
    - Breaking projects down into executable tasks with estimated completion times.

- Assigning priority levels (Low, Medium, High, Critical) and tracking completion percentages (0–100%).

4. Time & Financial Cost Logging:
    - Granular daily reporting of spent hours by employees on specific tasks.
    - Categorizing hours as billable or non-billable for accurate client billing and internal cost calculations.
5. Analytical Reporting & Performance Evaluation:
    - Calculating project cost variances by comparing accumulated labor costs (hours_spent * hourly_rate) against the project budget.
    - Aggregating workload distribution and identifying team over-utilization using complex multi-table joins, GROUP BY, and HAVING filters.

User groups served:

The system serves four primary stakeholder groups with distinct operational needs:

- Project Managers & Team Leads: Create projects, define tasks, assign staff, monitor schedule adherence, and track budget utilization to prevent cost overruns.
- Employees & Team Members: View assigned tasks, update work completion percentages, and record daily logged hours with descriptions of work performed.
- Executive Management & Department Heads: Review high-level departmental spending, project profitability, overall labor costs, and staff efficiency metrics.
- Database Administrators (DBA): Maintain structural integrity, manage default values, enforce check constraints, configure foreign key cascading rules, and control access permissions.

2. System Limitations

A candid evaluation of system limitations is an essential software engineering practice to establish operational boundaries and guide future enhancements:

1. Dependence on Human Data Entry Accuracy:
    - The reliability of cost variances, time logs, and progress reports relies strictly on employees submitting timely and accurate hours.
2. Absence of Native Payroll and Invoicing Modules:

- While the database calculates billable labor costs and total spent hours, it does not directly process monetary disbursements, compute taxes, or generate official financial tax invoices.

3. Lack of Integrated Communication Tools:
    - The system focuses strictly on data persistence and relational analytics; it does not contain native messaging platforms, task comment feeds, or automated email notification services.
4. Single-Currency Financial Assumption:
    - All budgets and hourly rates are evaluated in a single uniform currency, without real-time multi-currency exchange rate conversions
5. Static Hourly Rate Mechanics:
    - Employee hourly rates apply directly to historical log calculations unless detailed rate-history snapshot tables (SCD Type 2) are added in future iterations.

3. List of DML, DDL, and TCL Statements Used

The system utilizes all three foundational sub-languages of SQL to build, manipulate, and maintain data integrity:

1. Data Definition Language (DDL).

   DDL statements define, alter, and manage the underlying database schema and objects:
    - CREATE TABLE: Used to establish the 5 core relational tables (departments, employees, projects, tasks, time_Logs) with Primary Keys, Foreign Keys, NOT NULL, UNIQUE, DEFAULT, and CHECK constraints.
    - ALTER TABLE: Used to modify schema structures, add constraints, or adjust data types as requirements evolve.
    - DROP TABLE: Used to remove temporary objects (e.g., executing the required exercise of deriving a table from an existing one and subsequently dropping it)
2. Data Manipulation Language (DML).

   DML statements query, insert, update, and manage records within the relational structure:
    - SELECT: Used to execute retrieval operations, including:
        - Multi-table relational joins (INNER JOIN, LEFT OUTER JOIN)
        - Grouping and aggregation (SUM, AVG, COUNT, MIN, MAX, GROUP BY, HAVING).

- Row filtering criteria (WHERE, BETWEEN, IN, LIKE, IS NULL).
- Conditional logic (CASE, IFNULL) and scalar transformations (SUBSTR, LENGTH, ROUND, CONCAT, date extraction).
- Set operations (UNION, UNION ALL).

- INSERT: Used to populate tables with dataset records across multiple data types (VARCHAR, INT, DATE, DECIMAL).
- UPDATE: Used to modify existing row values, such as adjusting task priorities or updating completion percentages.
- DELETE: Used to remove records while observing referential integrity rules (ON DELETE CASCADE / ON DELETE SET NULL).

3. Transaction Control Language (TCL).

   TCL statements govern transaction boundaries to ensure ACID compliance and maintain data consistency across multi-step operations:

   - COMMIT: Permanently saves all transactional data changes to the database.
   - ROLLBACK: Undoes uncommitted changes if a constraint violation or transaction error occurs.
   - SAVEPOINT: Sets intermediate rollback points within complex batch operations.

2. Schema and ERD (5 points)

1. Database definition language (DBDL) to describe the tables and relationships.

Departments - Contains the list of company departments

department_id INT: PRIMARY KEY (PK), AUTO_INCREMENT

department_name VARCHAR(50): NOT NULL, UNIQUE

location VARCHAR(50): NOT NULL

budget_allocation DECIMAL(12,2) : CHECK(budget_allocation >0)

employees - Contains information about employees and their department assignments.

employee_id INT: PRIMARY KEY (PK) , AUTO_INCREMENT

first_name VARCHAR(50): NOT NULL

last_name VARCHAR(50): NOT NULL

email VARCHAR(50): NOT NULL, UNIQUE, CHECK(email like ‘%@%.%’)

hourly_rate DECIMAL(6,2): NOT NULL, CHECK(hourly_rate>=15.00)

department_id INT: FOREIGN KEY (FK) -> departments(department_id) ON DELETE SET NULL

manager_id INT: FK(Self-Reference) -> employees(employee_id) ON DELETE SET NULL

hire_date DATE: NOT NULL, DEFAULT(CURRENT_DATE)

is_active BOOLEAN: NOT NULL, DEFAULT 1

projects - Information about projects, their budgets, and statuses.

project_id INT: PRIMARY KEY (PK) , AUTO_INCREMENT

project_code VARCHAR(10): NOT NULL, UNIQUE (example, *'PRJ-2026-A')*

project_name VARCHAR(100): NOT NULL

client_name VARCHAR(100): NOT NULL

budget DECIMAL(12,2) : NOT NULL, CHECK(budget>0)

start_date DATE: NOT NULL

end_date DATE: CHECK(end_date IS NULL OR end_date >=start_date)

status VARCHAR(20): NOT NULL, DEFAULT 'Planning', CHECK(status IN ('Planning', 'In Progress', 'On Hold', 'Completed', 'Cancelled'))

lead_employee_id INT: FK -> employees(employee_id) ON DELETE SET NULL

tasks - Tasks within projects assigned to specific employees.

task_id INT: PRIMARY KEY (PK) , AUTO_INCREMENT

project_id INT: Foreign Key (FK) -> projects(project_id) ON DELETE CASCADE

assigned_to INT: Foreign Key (FK) -> employees(employee_id) ON DELETE SET NULL

task_title VARCHAR(150): NOT NULL

priority VARCHAR(10): NOT NULL, CHECK (priority IN (**'Low', 'Medium', 'High', 'Critical')**)

estimated_hours DECIMAL(5,2): NOT NULL, CHECK(estimated_hours > 0)

completion_pct INT: NOT NULL, DEFAULT 0, CHECK (completion_pct BETWEEN 0 AND 100)

created_at DATE: NOT NULL, DEFAULT(CURRENT_DATE)

time_logs - Employee reports on time spent completing tasks.

log_id INT: PRIMARY KEY (PK) , AUTO_INCREMENT

task_id INT: FK -> tasks(task_id) ON DELETE CASCADE

employee_id INT: FK -> employees(employee_id) ON DELETE CASCADE

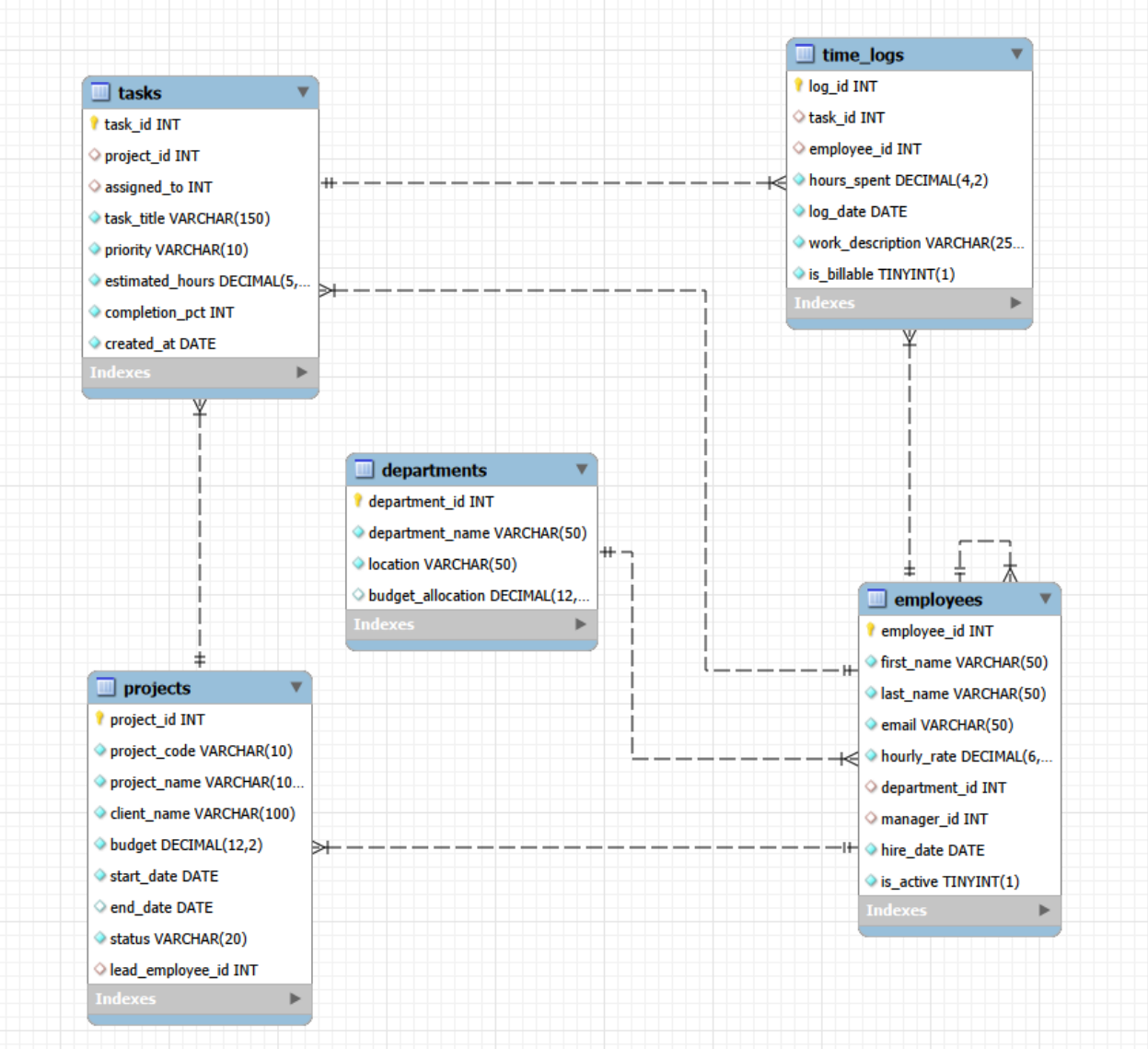
hours_spent DECIMAL(4,2): NOT NULL, CHECK (hours_spent > 0 AND hours_spent <= 24.00)

log_date DATE: NOT NULL, DEFAULT (CURRENT_DATE)

work_description VARCHAR(255) : NOT NULL

is_billable BOOLEAN: NOT NULL, DEFAULT 1

2***. Entity-Relationship Diagram (ERD).***



3. Tables (5 points)

Create 5 tables.

4. Defaults (2 points)

Add at least 2 different defaults.

5. Constraints (1 point)

Add a NOT NULL constraint.

6 Constraints (1 point)

Add at least 1 constraint using ENUM or CHECK constraint.

7. Constraints (10 points)

Each table should have a primary key and four of the tables should have foreign keys.

SCHEMA CREATION & DATA INTEGRITY RULES (SECTIONS 3–7)

```sql
-- create the database
DROP DATABASE IF EXISTS pm_db;
CREATE DATABASE pm_db;

-- select the datebase
USE pm_db;

-- 3. create the tables
CREATE TABLE departments
(
    department_id       INT             PRIMARY KEY     AUTO_INCREMENT,
    department_name     VARCHAR(50)     NOT NULL        UNIQUE,
    location            VARCHAR(50)     NOT NULL,
    budget_allocation   DECIMAL(12,2),
    CONSTRAINT departments_chk_budget_allocation
        CHECK (budget_allocation>0)
);
```

```sql
CREATE TABLE employees
(
    employee_id     INT             PRIMARY KEY     AUTO_INCREMENT,
    first_name      VARCHAR(50)     NOT NULL,
    last_name       VARCHAR(50)     NOT NULL,
    email           VARCHAR(50)     NOT NULL        UNIQUE,
    hourly_rate     DECIMAL(6,2)    NOT NULL,
    department_id   INT,
    manager_id      INT,
    hire_date       DATE            NOT NULL        DEFAULT(CURRENT_DATE),
    is_active       BOOLEAN         NOT NULL        DEFAULT 1,
    CONSTRAINT fk_manager_id
        FOREIGN KEY (manager_id) REFERENCES employees (employee_id)
        ON DELETE SET NULL,
    CONSTRAINT employees_chk_email
        CHECK(email like '%@%.%'),
    CONSTRAINT employees_chk_hourly_rate
        CHECK(hourly_rate>=15.00)
);
```

```sql
CREATE TABLE projects
(
    project_id          INT             PRIMARY KEY     AUTO_INCREMENT,
    project_code        VARCHAR(10)     NOT NULL        UNIQUE,
    project_name        VARCHAR(100)    NOT NULL,
    client_name         VARCHAR(100)    NOT NULL,
    budget              DECIMAL(12,2)   NOT NULL,
    start_date          DATE            NOT NULL,
    end_date            DATE,
    status              VARCHAR(20)     NOT NULL        DEFAULT 'Planning',
    lead_employee_id    INT,
    CONSTRAINT projects_chk_budget
        CHECK(budget>0),
    CONSTRAINT projects_chk_end_date
        CHECK(end_date IS NULL OR end_date>=start_date),
    CONSTRAINT projects_chk_status
        CHECK(status IN ('Planning', 'In Progress', 'On Hold', 'Completed', 'Cancelled'))
);
CREATE TABLE tasks
(
    task_id         INT             PRIMARY KEY     AUTO_INCREMENT,
    project_id      INT,
    assigned_to     INT,
    task_title      VARCHAR(150)    NOT NULL,
    priority        VARCHAR(10)     NOT NULL,
    estimated_hours DECIMAL(5,2)    NOT NULL,
    completion_pct  INT             NOT NULL        DEFAULT 0,
    created_at      date            NOT NULL        DEFAULT(CURRENT_DATE),
    CONSTRAINT tasks_chk_priority
        CHECK(priority IN('Low', 'Medium', 'High', 'Critical')),
    CONSTRAINT tasks_chk_estimated_hours
        CHECK(estimated_hours>0),
    CONSTRAINT tasks_chk_completion_pct
        CHECK(completion_pct BETWEEN 0  AND 100)
);
```

```sql
CREATE TABLE time_logs
(
    log_id              INT             PRIMARY KEY     AUTO_INCREMENT,
    task_id             INT,
    employee_id         INT,
    hours_spent         DECIMAL(4,2)    NOT NULL,
    log_date            DATE            NOT NULL    DEFAULT(CURRENT_DATE),
    work_description    VARCHAR(255)    NOT NULL,
    is_billable         BOOLEAN         NOT NULL    DEFAULT 1,
    CONSTRAINT time_logs_chk_hours_spent
        CHECK(hours_spent>0 AND hours_spent<=24.00)
);

-- add Foreign Key constraints via ALTER TABLE
ALTER TABLE employees
ADD CONSTRAINT employees_fk_departments
        FOREIGN KEY (department_id) REFERENCES departments (department_id)
        ON DELETE SET NULL;

ALTER TABLE projects
ADD CONSTRAINT projects_fk_employees
    FOREIGN KEY (lead_employee_id) REFERENCES employees(employee_id)
    ON DELETE SET NULL;

ALTER TABLE tasks
ADD CONSTRAINT tasks_fk_projects
    FOREIGN KEY (project_id) REFERENCES projects(project_id)
    ON DELETE CASCADE;
```

```sql
ALTER TABLE tasks
ADD CONSTRAINT tasks_fk_employees
    FOREIGN KEY (assigned_to) REFERENCES employees(employee_id)
    ON DELETE SET NULL;

ALTER TABLE time_logs
ADD CONSTRAINT time_logs_fk_tasks
    FOREIGN KEY (task_id) REFERENCES tasks(task_id)
    ON DELETE CASCADE;

ALTER TABLE time_logs
ADD CONSTRAINT time_logs_fk_employees
    FOREIGN KEY (employee_id) REFERENCES employees(employee_id)
    ON DELETE CASCADE;
```

8 Tables (4 points)

Create a table from another table, both the structure and the data.
Drop the created table.

```sql
-- 8.1 create a backup table 'active_employee_archive'
CREATE TABLE active_employee_archive
    SELECT employee_id, first_name, last_name, email, hourly_rate, department_id
    FROM employees
    WHERE is_active = 1;

-- 8.2 drop the created table
DROP TABLE active_employee_archive;
```

9. Data (5 points)

Insert at least ten records into three of the tables.
Use the following at least once: VARCHAR, INT, DATE, FLOAT/DECIMAL/DOUBLE

```sql
-- 9.1. insert departments (5 records)
INSERT INTO departments VALUES
    (DEFAULT, 'Executive', 'Columbus - HQ', 500000.00),
    (DEFAULT, 'Software Engineering', 'Cincinnati - Tech Hub', 1200000.00),
    (DEFAULT, 'Quality Assurance', 'Cincinnati - Tech Hub', 350000.00),
    (DEFAULT, 'Marketing & Design', 'Dayton - Creative Office', 450000.00),
    (DEFAULT, 'Human Resources', 'Columbus - HQ', 250000.00);

-- 9.2. insert employees (12 records: Managers first, then staff)
-- insert Executives & Department Managers (manager_id = NULL)
INSERT INTO employees VALUES
    (DEFAULT,'Alexander', 'Wright', 'a.wright@techcorp.com', 85.00, 1, NULL, '2020-01-15', 1),
    (DEFAULT,'Sophia', 'Chen', 's.chen@techcorp.com', 65.00, 2, 1, '2021-03-10', 1),
    (DEFAULT,'Marcus', 'Vance', 'm.vance@techcorp.com', 55.00, 3, 1, '2021-06-01', 1),
    (DEFAULT,'Emily', 'Watson', 'e.watson@techcorp.com', 50.00, 4, 1, '2022-02-15', 1),
    (DEFAULT,'Daniel', 'Miller', 'd.miller@techcorp.com', 45.00, 2, 2, '2022-05-10', 1),
    (DEFAULT,'Olivia', 'Taylor', 'o.taylor@techcorp.com', 42.00, 2, 2, '2023-01-20', 1),
    (DEFAULT,'Liam', 'Anderson', 'l.anderson@techcorp.com', 38.00, 3, 3, '2023-04-11', 1),
    (DEFAULT,'Ava', 'Martinez', 'a.martinez@techcorp.com', 35.00, 3, 3, '2023-08-01', 1),
    (DEFAULT,'Ethan', 'Thomas', 'e.thomas@techcorp.com', 40.00, 4, 4, '2024-01-15', 1),
    (DEFAULT,'Isabella', 'White', 'i.white@techcorp.com', 30.00, 5, 1, '2024-03-01', 1),
    (DEFAULT,'Lucas', 'Harris', 'l.harris@techcorp.com', 48.00, 2, 2, '2024-05-12', 1),
    (DEFAULT,'Mia', 'Clark', 'm.clark@techcorp.com', 25.00, 5, 1, '2025-02-01', 0);

-- 9.3. insert projects (10 records)
INSERT INTO projects VALUES
    (DEFAULT,'PRJ-2026-A', 'Enterprise ERP Portal', 'Nexus Global Corp', 150000.00, '2026-01-10', '2026-08-30', 'In Progress', 2),
    (DEFAULT,'PRJ-2026-B', 'Mobile Banking App Redesign', 'Apex Financial', 85000.00, '2026-02-01', '2026-06-15', 'In Progress', 2),
    (DEFAULT,'PRJ-2026-C', 'Cloud Infrastructure Migration', 'DataTech Systems', 120000.00, '2025-09-01', '2026-01-20', 'Completed', 1),
    (DEFAULT,'PRJ-2026-D', 'AI Customer Analytics Dashboard', 'RetailPulse Inc', 95000.00, '2026-03-15', '2026-11-30', 'In Progress', 2),
    (DEFAULT,'PRJ-2026-E', 'HR Automation Platform', 'Internal Operations', 40000.00, '2026-04-01', '2026-07-31', 'Planning', 4),
    (DEFAULT,'PRJ-2026-F', 'E-Commerce Payment Gateway', 'ShopQuick Inc', 110000.00, '2026-05-01', NULL, 'Planning', 2),
    (DEFAULT,'PRJ-2026-G', 'Security Audit & Compliance', 'Apex Financial', 30000.00, '2025-11-01', '2025-12-20', 'Completed', 3),
    (DEFAULT,'PRJ-2026-H', 'Brand Rebranding Campaign', 'Nexus Global Corp', 60000.00, '2026-02-15', '2026-05-31', 'On Hold', 4),
    (DEFAULT,'PRJ-2026-I', 'Legacy System Decommissioning', 'Internal Operations', 25000.00, '2026-01-01', '2026-03-01', 'Cancelled', 1),
    (DEFAULT,'PRJ-2026-J', 'IoT Smart Warehouse Portal', 'LogiTrans Logistics', 175000.00, '2026-06-01', '2026-12-31', 'Planning', 2);

-- 9.4. insert tasks (12 records)
INSERT INTO tasks VALUES
    (DEFAULT,1, 5, 'Database Schema Architecture Design', 'Critical', 40.00, 100, '2026-01-12'),
    (DEFAULT,1, 6, 'Backend REST API Implementation', 'High', 80.00, 65, '2026-01-20'),
    (DEFAULT,1, 7, 'API Integration Automated Testing', 'High', 35.00, 40, '2026-02-05'),
    (DEFAULT,2, 6, 'UI/UX Mobile Screen Wireframing', 'Medium', 30.00, 90, '2026-02-02'),
    (DEFAULT,2, 5, 'OAuth2 Authentication Setup', 'High', 25.00, 50, '2026-02-15'),
    (DEFAULT,3, 11, 'AWS S3 Bucket Migration', 'Medium', 50.00, 100, '2025-09-10'),
    (DEFAULT,4, 5, 'Machine Learning Model Integration', 'Critical', 100.00, 20, '2026-03-20'),
    (DEFAULT,4, 8, 'Data Pipeline Quality Verification', 'High', 45.00, 10, '2026-03-25'),
    (DEFAULT,5, 9, 'Employee Onboarding Portal UI', 'Low', 50.00, 0, '2026-04-02'),
    (DEFAULT,8, 9, 'Marketing Graphic Assets Creation', 'Medium', 35.00, 30, '2026-02-18'),
    (DEFAULT,1, 11, 'GraphQL Endpoint Optimization', 'High', 20.00, 15, '2026-03-01'),
    (DEFAULT,7, 8, 'Penetration Testing & Vulnerability Scan', 'Critical', 40.00, 100, '2025-11-05');
```

```sql
-- 9.5. insert time_Logs (12 records)
INSERT INTO time_logs VALUES
    (DEFAULT,1, 5, 8.00, '2026-01-15', 'Created relational ERD and defined foreign keys', 1),
    (DEFAULT,1, 5, 7.50, '2026-01-16', 'Optimized DDL indexes and check constraints', 1),
    (DEFAULT,2, 6, 6.00, '2026-01-22', 'Developed User Auth endpoints in Node.js', 1),
    (DEFAULT,2, 6, 8.00, '2026-01-23', 'Connected REST APIs with PostgreSQL driver', 1),
    (DEFAULT,3, 7, 5.00, '2026-02-08', 'Wrote integration unit tests with Jest', 1),
    (DEFAULT,4, 6, 4.50, '2026-02-04', 'Designed Figma mobile mockups for client', 1),
    (DEFAULT,5, 5, 6.00, '2026-02-17', 'Configured JWT tokens and refresh mechanisms', 1),
    (DEFAULT,6, 11, 8.00, '2025-09-15', 'Migrated 5TB S3 bucket data to AWS West', 1),
    (DEFAULT,7, 5, 7.00, '2026-03-22', 'Trained initial Scikit-Learn prediction model', 1),
    (DEFAULT,8, 8, 4.00, '2026-03-28', 'Validated clean sample data ingestion', 1),
    (DEFAULT,10, 9, 3.50, '2026-02-20', 'Internal meeting regarding logo rebranding', 0),
    (DEFAULT,12, 8, 8.00, '2025-11-10', 'Executed OWASP security audit suite', 1);
```

10 Data (1 point)

Update at least three records.

```sql
-- update 10.1: Increase hourly rate by 10% for Software Engineering employees
UPDATE employees SET hourly_rate = hourly_rate * 1.1
WHERE department_id = (
SELECT department_id FROM departments
WHERE department_name = "Software Engineering");

-- update 10.2: Update completion status and percentage for Task #2
UPDATE tasks
SET completion_pct = 100, priority = 'Medium'
WHERE task_id = 2;

-- update 10.3: Complete project PRJ-2026-B and record today's date as end_dated
UPDATE projects
SET status = 'Completed',
    end_date = CURRENT_DATE()
WHERE project_code = "PRJ-2026-B";
```

11 Aggregates (6 points)

GROUP BY clauses, and queries with:

(1) sum and average

```sql
-- 11.1. SUM and AVG with GROUP BY
-- Business Purpose: Evaluates total and average financial allocation across project statuses.
SELECT status,
    round(sum(budget),2) AS total_budget,
    round(avg(budget),2) AS avg_budget
FROM projects
GROUP BY status;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

status	total_budget	avg_budget
In Progress	245000.00	122500.00
Completed	235000.00	78333.33
Planning	325000.00	108333.33
On Hold	60000.00	60000.00
Cancelled	25000.00	25000.00

(2) count

```sql
-- 11.2. COUNT with GROUP BY
-- Business Purpose: Calculates total headcount assigned to each organizational department.
SELECT d.department_name,
    COUNT(e.employee_id) AS total_employees
FROM employees e
JOIN departments d USING (department_id)
GROUP BY d.department_id, d.department_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

department_name	total_employees
Executive	1
Human Resources	2
Marketing & Design	2
Quality Assurance	3
Software Engineering	4

(3) min and max.

```sql
-- 11.3. MIN and MAX with GROUP BY
-- Business Purpose: Identifies the compensation spread (minimum and maximum hourly rates) per department.
SELECT d.department_name,
    MAX(e.hourly_rate) AS max_hourly_rate,
    MIN(e.hourly_rate) AS min_hourly_rate
FROM employees e
JOIN departments d USING (department_id)
GROUP BY d.department_id, d.department_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

department_name	max_hourly_rate	min_hourly_rate
Executive	85.00	85.00
Human Resources	30.00	25.00
Marketing & Design	50.00	40.00
Quality Assurance	55.00	35.00
Software Engineering	71.50	46.20

Use GROUP BY and HAVING.

```sql
-- HAVING clause with GROUP BY
-- Business Purpose: Filters for high-effort projects that accumulated over 10 total logged hours.
SELECT p.project_id,
    p.project_name,
    SUM(tl.hours_spent) AS total_hours_spent
FROM tasks t
JOIN time_logs tl USING (task_id)
JOIN projects p USING (project_id)
GROUP BY p.project_id, p.project_name
HAVING total_hours_spent > 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

project_id	project_name	total_hours_spent
1	Enterprise ERP Portal	34.50
2	Mobile Banking App Redesign	10.50
4	AI Customer Analytics Dashboard	11.00

12 Queries (26 points)

Queries to restrict rows using criteria and compound criteria:

a) Range criteria (BETWEEN)

```
285     -- 12.a) Range Criteria (BETWEEN)
286     -- Retrieves employees earning an hourly rate within the $40.00 to $60.00 range.
287 •   SELECT employee_id, first_name, last_name, hourly_rate
288     FROM employees
289     WHERE hourly_rate BETWEEN 40.00 AND 60.00;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	first_name	last_name	hourly_rate
3	Marcus	Vance	55.00
4	Emily	Watson	50.00
5	Daniel	Miller	49.50
6	Olivia	Taylor	46.20
9	Ethan	Thomas	40.00
11	Lucas	Harris	52.80

b) Using IN operator

```
291     -- 12.b) IN Operator
292     -- Filters active projects matching specific strategic workflow states.
293 •   SELECT project_code, project_name, status
294     FROM projects
295     WHERE status IN ('In Progress', 'Planning', 'On Hold')
296     ORDER BY status;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

project_code	project_name	status
PRJ-2026-A	Enterprise ERP Portal	In Progress
PRJ-2026-D	AI Customer Analytics Dashboard	In Progress
PRJ-2026-H	Brand Rebranding Campaign	On Hold
PRJ-2026-E	HR Automation Platform	Planning
PRJ-2026-F	E-Commerce Payment Gateway	Planning
PRJ-2026-J	IoT Smart Warehouse Portal	Planning

c) Using LIKE operator with wildcards

```
298     -- 12.c) LIKE Operator (Compound OR logic)
299     -- Searches for employees whose last name starts with 'W' OR who possess corporate email accounts.
300 •   SELECT employee_id, first_name, last_name, email
301     FROM employees
302     WHERE last_name LIKE 'W%'
303         OR email LIKE '%@techcorp.com';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	first_name	last_name	email
1	Alexander	Wright	a.wright@techcorp.com
2	Sophia	Chen	s.chen@techcorp.com
3	Marcus	Vance	m.vance@techcorp.com
4	Emily	Watson	e.watson@techcorp.com
5	Daniel	Miller	d.miller@techcorp.com
6	Olivia	Taylor	o.taylor@techcorp.com
7	Liam	Anderson	l.anderson@techcorp.com
8	Ava	Martinez	a.martinez@techcorp.com
9	Ethan	Thomas	e.thomas@techcorp.com
10	Isabella	White	i.white@techcorp.com
11	Lucas	Harris	l.harris@techcorp.com
12	Mia	Clark	m.clark@techcorp.com

d) Use column alias

```
305     -- 12.d) Column Aliases & Sorting
306     -- Displays full staff names alongside custom-labeled compensation columns.
307 •   SELECT CONCAT(first_name,' ',last_name) AS 'Full Name',
308         hourly_rate AS 'Hourly Rate ($)'
309     FROM employees
310     ORDER BY last_name, first_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Full Name	Hourly Rate ($)
Liam Anderson	38.00
Sophia Chen	71.50
Mia Clark	25.00
Lucas Harris	52.80
Ava Martinez	35.00
Daniel Miller	49.50
Olivia Taylor	46.20
Ethan Thomas	40.00
Marcus Vance	55.00
Emily Watson	50.00
Isabella White	30.00
Alexander Wr...	85.00

e) Perform basic arithmetic operations

```
312     -- 12.e) Basic Arithmetic Operations
313     -- Projects expected financial expense by multiplying estimated hours by the assigned employee's rate.
314 •   SELECT t.task_id, t.task_title, e.hourly_rate, t.estimated_hours,
315         ROUND( e.hourly_rate * t.estimated_hours, 2) AS estimated_task_cost
316     FROM tasks t
317     JOIN employees e ON t.assigned_to = e.employee_id
318     ORDER BY t.task_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

task_id	task_title	hourly_rate	estimated_hours	estimated_task_cost
1	Database Schema Architecture Design	49.50	40.00	1980.00
2	Backend REST API Implementation	46.20	80.00	3696.00
3	API Integration Automated Testing	38.00	35.00	1330.00
4	UI/UX Mobile Screen Wireframing	46.20	30.00	1386.00
5	OAuth2 Authentication Setup	49.50	25.00	1237.50
6	AWS S3 Bucket Migration	52.80	50.00	2640.00
7	Machine Learning Model Integration	49.50	100.00	4950.00
8	Data Pipeline Quality Verification	35.00	45.00	1575.00
9	Employee Onboarding Portal UI	40.00	50.00	2000.00
10	Marketing Graphic Assets Creation	40.00	35.00	1400.00
11	GraphQL Endpoint Optimization	52.80	20.00	1056.00
12	Penetration Testing & Vulnerability Scan	35.00	40.00	1400.00

f) Concatenate when reporting data

```
320     -- 12.f) String Concatenation
321     -- Combines discrete first and last name fields into a unified full name string.
322 •   SELECT CONCAT(first_name,' ',last_name) AS 'Full Name'
323     FROM employees
324     ORDER BY last_name, first_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Full Name
Liam Anderson
Sophia Chen
Mia Clark
Lucas Harris
Ava Martinez
Daniel Miller
Olivia Taylor
Ethan Thomas
Marcus Vance
Emily Watson
Isabella White
Alexander Wright

g) Use DISTINCT keyword

```
326     -- 12.g) DISTINCT Keyword
327     -- Extracts a unique list of corporate clients without duplication.
328 •   SELECT DISTINCT client_name
329     FROM projects
330     ORDER BY client_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

client_name
Apex Financial
DataTech Systems
Internal Operations
LogiTrans Logistics
Nexus Global Corp
RetailPulse Inc
ShopQuick Inc

h) Order the output

```
332     -- 12.h) Sorting Output (ORDER BY DESC)
333     -- Orders the portfolio from highest to lowest allocated budget.
334 •   SELECT project_code, project_name, budget
335     FROM projects
336     ORDER BY budget DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

project_code	project_name	budget
PRJ-2026-J	IoT Smart Warehouse Portal	175000.00
PRJ-2026-A	Enterprise ERP Portal	150000.00
PRJ-2026-C	Cloud Infrastructure Migration	120000.00
PRJ-2026-F	E-Commerce Payment Gateway	110000.00
PRJ-2026-D	AI Customer Analytics Dashboard	95000.00
PRJ-2026-B	Mobile Banking App Redesign	85000.00
PRJ-2026-H	Brand Rebranding Campaign	60000.00
PRJ-2026-E	HR Automation Platform	40000.00
PRJ-2026-G	Security Audit & Compliance	30000.00
PRJ-2026-I	Legacy System Decommissioning	25000.00

i) List all the records with NULL values

```
338     -- 12.i) NULL Values Handling
339     -- Identifies ongoing projects lacking an end date OR team members without assigned managers.
340 •   SELECT p.project_code, p.project_name, p.end_date, e.manager_id
341     FROM projects p
342     LEFT JOIN employees e ON p.lead_employee_id = e.employee_id
343     WHERE p.end_date IS NULL
344         OR e.manager_id IS NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

project_code	project_name	end_date	manager_id
PRJ-2026-C	Cloud Infrastructure Migration	2026-01-20	NULL
PRJ-2026-F	E-Commerce Payment Gateway	NULL	1
PRJ-2026-I	Legacy System Decommissioning	2026-03-01	NULL

j) Extract the month from the date

```
346     -- 12.j) Date Extraction (MONTH)
347     -- Extracts the numerical hire month for employee tenure analytics.
348 •   SELECT CONCAT(first_name, ' ',last_name) AS full_name,
349         hire_date,
350         MONTH(hire_date) AS hire_month
351     FROM employees
352     ORDER BY hire_month;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	hire_date	hire_month
Alexander Wright	2020-01-15	1
Olivia Taylor	2023-01-20	1
Ethan Thomas	2024-01-15	1
Emily Watson	2022-02-15	2
Mia Clark	2025-02-01	2
Sophia Chen	2021-03-10	3
Isabella White	2024-03-01	3
Liam Anderson	2023-04-11	4
Daniel Miller	2022-05-10	5
Lucas Harris	2024-05-12	5
Marcus Vance	2021-06-01	6
Ava Martinez	2023-08-01	8

k) Use SUBSTR to pull out something

```sql
-- 12.k) SUBSTR Function
-- Extracts the leading 3-character prefix from unique project codes.
SELECT project_code,
    SUBSTR(project_code, 1 ,3) AS project_prefix
FROM projects;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

project_code	project_prefix
PRJ-2026-A	PRJ
PRJ-2026-B	PRJ
PRJ-2026-C	PRJ
PRJ-2026-D	PRJ
PRJ-2026-E	PRJ
PRJ-2026-F	PRJ
PRJ-2026-G	PRJ
PRJ-2026-H	PRJ
PRJ-2026-I	PRJ
PRJ-2026-J	PRJ

l) Create a column that provides the length of one of your columns

```sql
-- 12.l) String Length (LENGTH)
-- Calculates character length of project titles for data validation.
SELECT project_name,
    LENGTH(project_name) AS project_name_length
FROM projects
ORDER BY project_name_length DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

project_name	project_name_length
AI Customer Analytics Dashboard	31
Cloud Infrastructure Migration	30
Legacy System Decommissioning	29
Mobile Banking App Redesign	27
Security Audit & Compliance	27
E-Commerce Payment Gateway	26
IoT Smart Warehouse Portal	26
Brand Rebranding Campaign	25
HR Automation Platform	22
Enterprise ERP Portal	21

m) Round output from one of your columns to one digit

```sql
-- 12.m) Numeric Rounding (ROUND to 1 decimal place)
-- Computes average organizational hourly rate rounded strictly to 1 decimal place.
SELECT ROUND(AVG(hourly_rate), 1) AS avg_hourly_rate_rounded
FROM employees;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

avg_hourly_rate_rounded
48.2

13 Queries (8 points)

Queries that show selection

a) Use a CASE statement

```sql
-- 13.a) CASE Statement
-- Categorizes project portfolio into budget tiers for executive decision-making.
SELECT project_name, budget,
    CASE
        WHEN budget > 100000 THEN 'High Budget'
        WHEN budget BETWEEN 50000 AND 100000 THEN 'Medium Budget'
        ELSE 'Low Budget'
    END AS budget_category
FROM  projects
ORDER BY budget DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

project_name	budget	budget_category
IoT Smart Warehouse Portal	175000.00	High Budget
Enterprise ERP Portal	150000.00	High Budget
Cloud Infrastructure Migration	120000.00	High Budget
E-Commerce Payment Gateway	110000.00	High Budget
AI Customer Analytics Dashboard	95000.00	Medium Budget
Mobile Banking App Redesign	85000.00	Medium Budget
Brand Rebranding Campaign	60000.00	Medium Budget
HR Automation Platform	40000.00	Low Budget
Security Audit & Compliance	30000.00	Low Budget
Legacy System Decommissioning	25000.00	Low Budget

b) Use IF or IFNULL

```sql
-- 13.b) IFNULL Function
-- Replaces unassigned project end dates with descriptive operational statuses.
SELECT project_code, status,
    IFNULL(end_date, 'In Progress / Ongoing') AS actual_end_date
FROM projects;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

project_code	status	actual_end_date
PRJ-2026-A	In Progress	2026-08-30
PRJ-2026-B	Completed	2026-10-18
PRJ-2026-C	Completed	2026-01-20
PRJ-2026-D	In Progress	2026-11-30
PRJ-2026-E	Planning	2026-07-31
PRJ-2026-F	Planning	In Progress / Ongoing
PRJ-2026-G	Completed	2025-12-20
PRJ-2026-H	On Hold	2026-05-31
PRJ-2026-I	Cancelled	2026-03-01
PRJ-2026-J	Planning	2026-12-31

14 Joins (12 points)

Queries to join multiple tables together based on relationships:

a) Query that joins 3 tables and limits output using a WHERE clause

```sql
-- 14.a) Join 3 Tables with Filtering
-- Connects projects, tasks, and employees to review high-priority task assignments.
SELECT p.project_name, t.task_title, t.priority,
    CONCAT(e.first_name, ' ', e.last_name) AS assigned_employee
FROM tasks t
JOIN projects p USING(project_id)
JOIN employees e ON t.assigned_to = e.employee_id
WHERE t.priority IN ('Critical', 'High')
ORDER BY t.priority;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

project_name	task_title	priority	assigned_employee
Enterprise ERP Portal	Database Schema Architecture Design	Critical	Daniel Miller
AI Customer Analytics Dashboard	Machine Learning Model Integration	Critical	Daniel Miller
Security Audit & Compliance	Penetration Testing & Vulnerability Scan	Critical	Ava Martinez
Enterprise ERP Portal	API Integration Automated Testing	High	Liam Anderson
Mobile Banking App Redesign	OAuth2 Authentication Setup	High	Daniel Miller
AI Customer Analytics Dashboard	Data Pipeline Quality Verification	High	Ava Martinez
Enterprise ERP Portal	GraphQL Endpoint Optimization	High	Lucas Harris

b) Query with a LEFT OUTER JOIN

```
409     -- 14.b) LEFT OUTER JOIN
410     -- Displays all employees regardless of whether they currently have assigned tasks.
411 •   SELECT CONCAT(e.first_name, ' ', e.last_name) AS full_name,
412         t.task_title
413     FROM employees e
414     LEFT JOIN tasks t ON e.employee_id = t.assigned_to
415     ORDER BY full_name;
416
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	task_title
Alexander Wright	NULL
Ava Martinez	Data Pipeline Quality Verification
Ava Martinez	Penetration Testing & Vulnerability Scan
Daniel Miller	Database Schema Architecture Design
Daniel Miller	OAuth2 Authentication Setup
Daniel Miller	Machine Learning Model Integration
Emily Watson	NULL
Ethan Thomas	Employee Onboarding Portal UI
Ethan Thomas	Marketing Graphic Assets Creation
Isabella White	NULL
Liam Anderson	API Integration Automated Testing
Lucas Harris	AWS S3 Bucket Migration
Lucas Harris	GraphQL Endpoint Optimization
Marcus Vance	NULL
Mia Clark	NULL
Olivia Taylor	Backend REST API Implementation
Olivia Taylor	UI/UX Mobile Screen Wireframing
Sophia Chen	NULL

c) Query with an INNER JOIN

```
417     -- 14.c) INNER JOIN
418     -- Links employee records with department entities to output organizational roles.
419 •   SELECT CONCAT(e.first_name, ' ', e.last_name) AS full_name,
420         d.department_name, e.hourly_rate
421     FROM employees e
422     INNER JOIN departments d USING(department_id)
423     ORDER BY d.department_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	department_name	hourly_rate
Alexander Wright	Executive	85.00
Isabella White	Human Resources	30.00
Mia Clark	Human Resources	25.00
Emily Watson	Marketing & Design	50.00
Ethan Thomas	Marketing & Design	40.00
Marcus Vance	Quality Assurance	55.00
Liam Anderson	Quality Assurance	38.00
Ava Martinez	Quality Assurance	35.00
Sophia Chen	Software Engineering	71.50
Daniel Miller	Software Engineering	49.50
Olivia Taylor	Software Engineering	46.20
Lucas Harris	Software Engineering	52.80

d) Query using a set operation (UNION, UNION ALL)

```
425     -- 14.d) Set Operation (UNION)
426     -- Combines client names and department titles into a single unified directory.
427 •   SELECT client_name AS entity_name, 'Client' AS entity_type
428     FROM projects
429     UNION
430     SELECT department_name AS entity_name, 'Department' AS entity_type
431     FROM departments
432     ORDER BY entity_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

entity_name	entity_type
Apex Financial	Client
DataTech Systems	Client
Executive	Department
Human Resources	Department
Internal Operations	Client
LogiTrans Logistics	Client
Marketing & Design	Department
Nexus Global Corp	Client
Quality Assurance	Department
RetailPulse Inc	Client
ShopQuick Inc	Client
Software Engineering	Department

15 Properly submitted (8 points)

1. Project description
2. Entity-relationship diagram and DBDL
3. Screenshots, labeled with item number saved to a Word or other document
4. SQL script(s) with code clearly documented with comments showing which code goes with which question